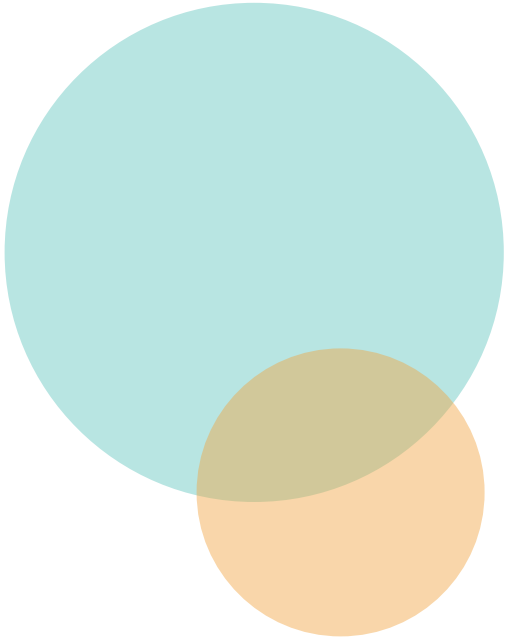
TP 1

Initiation à Python pour ML

Jupyter, NumPy, Pandas, visualisation et premier modèle de Machine Learning



Objectifs et plan du TP

Partie 1

Jupyter et bases de Python

- Lancer Jupyter, comprendre un notebook
- Variables, types, listes, dictionnaires
- Conditions et boucles
- Fonctions et import de bibliothèques

Partie 2

Manipulation de données

- NumPy : tableaux, indexation, opérations, agrégations
- Pandas : CSV, sélection, filtre, tri, groupby
- Valeurs manquantes
- NumPy ou Pandas : que choisir ?

Partie 3

Visualisation et premier ML

- Matplotlib et Seaborn : courbes, histogrammes, nuages, corrélations
- Scikit-learn : jeu Iris, features et target
- Séparation train/test
- Premier modèle simple

Lancer Jupyter

Jupyter est l'environnement dans lequel nous écrirons tout le TP : du code, du texte et des résultats au même endroit.

```
# 1. Installer (une seule fois)
pip install jupyterlab numpy pandas
pip install matplotlib seaborn scikit-learn

# 2. Lancer l'interface moderne
jupyter lab

# ou l'interface classique
jupyter notebook
```

Que se passe-t-il ?

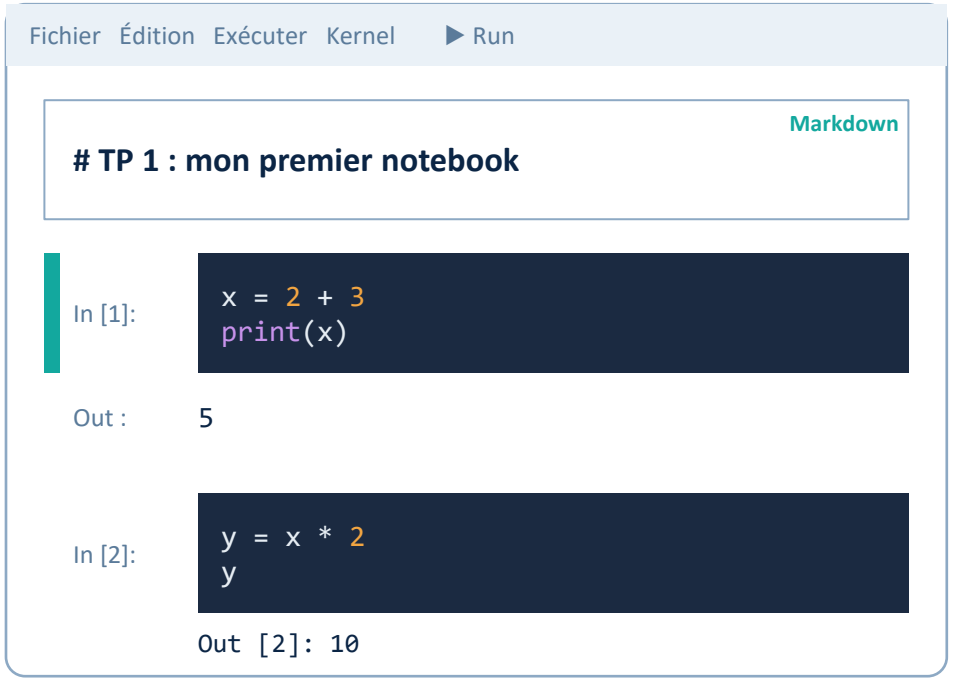
- Le terminal affiche une adresse du type http://localhost:8888
- Le navigateur s'ouvre tout seul sur la page d'accueil
- Avec Anaconda : lancer JupyterLab depuis Anaconda Navigator

Créer votre notebook

- Dans le navigateur : bouton + (Launcher), puis Notebook Python 3
- Renommer le fichier (clic droit, Rename) : il se termine par .ipynb
- Enregistrer souvent avec Ctrl + S

La structure d'un notebook

Un notebook est une suite de cellules. Chaque cellule est soit du code, soit du texte.



Les éléments à connaître

- Cellule code : on l'exécute, le résultat s'affiche dessous
- Cellule Markdown : titres et explications (# Titre, **gras**)
- In [n] : numéro d'ordre d'exécution
- Kernel : le moteur Python qui garde les variables en mémoire

Raccourcis essentiels

- Maj + Entrée : exécuter la cellule
- Échap puis A ou B : cellule au-dessus ou en dessous
- Échap puis M ou Y : Markdown ou code
- Échap puis D D : supprimer la cellule

Variables et types

Une variable est un nom qui désigne une valeur. Python devine le type tout seul : pas besoin de le déclarer.

```python
nom = "Amine"
age = 20
moyenne = 14.5
inscrit = True
print(type(nom), type(age))
print(type(moyenne), type(inscrit))
print(f"{nom} a {age} ans, moyenne {moyenne}")
```

Résultat

```
<class 'str'> <class 'int'>
<class 'float'> <class 'bool'>
Amine a 20 ans, moyenne 14.5
```

Opérations de base : + - * / // % **

Exemple : 7 // 2 donne 3 (division entière), 7 % 2 donne 1 (reste), 2 ** 3 donne 8 (puissance).

À retenir

- int : entier, float : décimal (avec un point)
- str : texte entre guillemets, bool : True ou False
- type(x) donne le type ; f"...{x}..." insère des valeurs dans un texte

Les listes

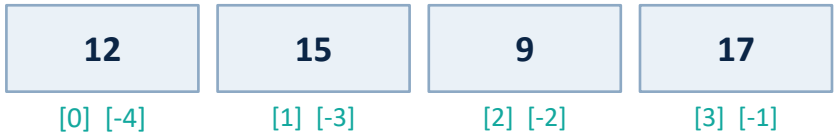
Une liste regroupe plusieurs valeurs ordonnées et modifiables. Le premier élément a l'indice 0.

```python
notes = [12, 15, 9, 17]
print(notes[:])
print(notes[0], notes[-1])
print(notes[1:3])
notes.append(14)
print(len(notes), sum(notes) / len(notes))
```

Résultat

```
[12, 15, 9, 17]
12 17
[15, 9]
5 13.4
```

Schéma de la liste notes (avant append)



Fonctionnement avec le *slicing*

Le principe reste le même : la borne de début est **incluse** et la borne de fin est **exclue**.

- notes[-3:-1] donne **[15, 9]** *(part de l'index -3 jusqu'à l'index -1 exclu)*
- notes[:-1] donne **[12, 15, 9]** *(prend tout sauf le dernier élément)*

sequence[début : fin : pas]

- **début** : L'index où commence l'extraction (**inclus**).
- **fin** : L'index où s'arrête l'extraction (**exclu**).
- **pas** *(optionnel)* : Le saut entre chaque élément (par défaut égal à 1).

À retenir

- notes[0] : premier élément ; notes[-1] : dernier
- notes[1:3] : tranche (indice 1 inclus, 3 exclu)
- append ajoute à la fin ; len donne la taille

Opérations Avancées et Méthodes Principales

Méthodes et Tranches (Slicing) sur les Listes

Ajout et Suppression :

.append(x) : Ajoute x à la fin de la liste.

```
fruits = ["pomme", "banane"]
fruits.append("orange")
# ["pomme", "banane", "orange"]
```

.insert(i, x) : Insère x à l'index i.

```
fruits.insert(1, "kiwi")
# ["pomme", "kiwi", "banane", "orange"]
```

.remove(x) : Supprime la première occurrence de x.

```
fruits.remove("banane")
# ["pomme", "kiwi", "orange"]
```

.pop(i) : Retire et renvoie l'élément situé à l'index i.

```
elem = fruits.pop(0)
# elem = "pomme", fruits = ["kiwi", "orange"]
```

Opérations Avancées et Méthodes Principales

Les Méthodes et Tranches (Slicing) sur les Listes

Découpage (Slicing) :

Syntaxe : liste[début:fin:pas]

```
nombres = [10, 20, 30, 40, 50]
# Slicing basique [début:fin]
nombres[1:3]
# Renvoie [20, 30]
# Avec un pas [début:fin:pas]
nombres[0:5:2]
# Renvoie [10, 30, 50]
```

Opérations Avancées et Méthodes Principales

Les Méthodes et Tranches (Slicing) sur les Listes

Tri et Reconstitution :

.sort() : Trie la liste sur place (ordre croissant).

```
#Réorganisez les éléments directement dans la liste
nums = [3, 1, 4, 2]
nums.sort()
# nums devient [1, 2, 3, 4]
```

.reverse() : Inverse l'ordre des éléments de la liste.

```
nums.reverse()
# nums devient [4, 3, 2, 1]
```

Les dictionnaires

Un dictionnaire associe une clé à une valeur. On retrouve une valeur par sa clé, pas par sa position.

```python
etudiant = {"nom": "Sara", "age": 21}
etudiant["filiere"] = "Maths“

print(etudiant["nom"])
print(list(etudiant.keys()))

for valeur in etudiant.values():
    print(valeur)


for cle, valeur in etudiant.items():
    print(cle, "->", valeur)
```

Résultat

```
Sara
['nom', 'age', 'filiere']
Sara
21
Maths
nom -> Sara
age -> 21
filiere -> Maths
```

Liste ou dictionnaire ?

Liste : une suite de valeurs de même nature (des notes). Dictionnaire : les champs d'un objet (nom, âge, filière).

À retenir

- Syntaxe : {clé: valeur, ...}
- d[clé] lit ou modifie ; une clé absente est ajoutée
- .items() donne les couples (clé, valeur) pour une boucle

Les conditions : if, elif, else

On exécute un bloc de code seulement si une condition est vraie. L'indentation (4 espaces) délimite le bloc.

```python
note = 13
if note >= 16:
    print("Très bien")
elif note >= 10:
    print("Admis")
else:
    print("Ajourné")
```

Résultat

```
Admis
```

Piège classique

= affecte une valeur (note = 13), == compare deux valeurs (note == 13). Oublier les deux-points ou l'indentation (indentation: désigne le fait d'ajouter un **espace ou un retrait au début d'une ligne de code)** provoque une erreur de syntaxe.

À retenir

- Comparaisons : `==` `!=` `<` `<=` `>` `>=`
- Combiner : and, or, not
- Une seule branche s'exécute : la première condition vraie

Les boucles : for et while

Une boucle répète un bloc d'instructions. for parcourt une séquence, while répète tant qu'une condition est vraie. La boucle *for* en Python nécessite une **séquence d'éléments** (une liste, un ensemble ou une suite de nombres) sur laquelle elle peut naviguer un par un, **range(3)** génère une séquence automatique de **3 nombres** : 0, 1 et 2 Contrairement à la boucle for (utilisée quand on connaît le nombre exact de tours ou la séquence à parcourir), la boucle while est idéale lorsque le nombre de répétitions dépend d'un événement dynamique (Condition).

Python

```python
for i in range(3):
    print("tour", i)

somme, n = 0, 1
while n <= 4:
    somme += n
    n += 1
print(somme)

carres = [x ** 2 for x in range(5)]
print(carres)
```

Résultat

```
tour 0
tour 1
tour 2
10
[0, 1, 4, 9, 16]
```

À retenir

- range(3) donne 0, 1, 2 (le 3 est exclu)
- while : penser à modifier la variable, sinon boucle infinie
- [expr for x in ...] : liste construite en une ligne

break : sortir de la boucle

break arrête immédiatement la boucle, même s'il restait des éléments à parcourir. Le programme reprend après la boucle.

Python

```python
notes = [12, 15, -1, 9, 17]
for n in notes:
    if n < 0:
        print("Valeur invalide : arrêt")
        break
    print("note lue :", n)
print("Fin de la boucle")
```

Parcours de la liste notes



Résultat

```
note lue : 12
note lue : 15
Valeur invalide : arrêt
Fin de la boucle
```

À retenir

- Dès que n < 0, break quitte la boucle
- Les valeurs 9 et 17 ne sont jamais lues
- Fonctionne aussi dans while (while True + break)

continue : passer au tour suivant

continue abandonne le tour en cours, saute le reste du bloc, puis reprend la boucle avec l'élément suivant.

```python
notes = [12, 15, -1, 9, 17]
for n in notes:
    if n < 0:
        continue
    print("note lue :", n)
print("Fin de la boucle")
```

Python

Résultat

```
note lue : 12
note lue : 15
note lue : 9
note lue : 17
Fin de la boucle
```

Parcours de la liste notes



À retenir

- La valeur -1 est ignorée : print n'est pas exécuté pour elle
- La boucle se poursuit avec 9 puis 17
- Utile pour écarter les valeurs invalides

break, continue et pass : les différences

Même boucle, même liste [12, 15, -1, 9, 17] : seul le mot-clé placé dans le if change.

break

Quitte la boucle

```
for n in [12, 15, -1, 9, 17]:
    if n < 0:
        break
    print(n, end=" ")
```

Affiche :

```
12 15
```

continue

Passe au tour suivant

```
for n in [12, 15, -1, 9, 17]:
    if n < 0:
        continue
    print(n, end=" ")
```

Affiche :

```
12 15 9 17
```

pass

Ne fait rien, la suite du bloc s'exécute

```
for n in [12, 15, -1, 9, 17]:
    if n < 0:
        pass
    print(n, end=" ")
```

Affiche :

```
12 15 -1 9 17
```

Définir et appeler une fonction

Une fonction regroupe du code réutilisable : on la définit une fois avec def, puis on l'appelle autant de fois que voulu.

Python

```
def moyenne(valeurs, arrondi=2):
    """Retourne la moyenne d'une liste avec une précision
décimales."""
    return round(sum(valeurs) / len(valeurs), arrondi)

m = moyenne([12, 15, 9, 17])
print(m)
print(moyenne([12, 15, 9, 17], arrondi=0))
```

Résultat

```
13.25
13.0
```

Ordre d'exécution

1. def enregistre la fonction (rien ne s'affiche). 2. L'appel moyenne([...]) exécute son corps. 3. La valeur renvoyée est stockée dans m.

À retenir

- def nom(paramètres): ... définit la fonction
- return renvoie le résultat à l'appelant
- arrondi=2 est une valeur par défaut, modifiable à l'appel

À retenir :

→ `#` : introduit un commentaire sur une ligne.

→ `"""..."""` : permet notamment d'écrire une docstring (docstring: sert à documenter une fonction, une classe ou un module. Contrairement à un commentaire, c'est une chaîne de caractères que Python conserve et rend accessible. On peut consulter cette documentation avec : help(calculer_moyenne)).

Définir et appeler une fonction

En pratique...

Pass ,en pratique,est utiliser pour esquisser une fonction ou une classe à compléter

```python
def calculer_moyenne(valeurs):
    pass            # à écrire plus tard
```

Sans pass : erreur

Écrire seulement « def f(x): » suivi d'une ligne vide provoque une IndentationError (« expected an indented block »). Ajouter pass règle le problème.

Importer une bibliothèque

Une bibliothèque est une boîte à outils déjà écrite. On l'importe avant de s'en servir.

```python
import math
import numpy as np
from math import sqrt

print(math.pi)
print(sqrt(16))
print(np.mean([1, 2, 3]))
```

Résultat

```
3.141592653589793
4.0
2.0
```

Erreur ModuleNotFoundError ?

La bibliothèque n'est pas installée : **pip install nom_du_paquet**, puis relancer le kernel.

À retenir

- import math : on écrit ensuite math.pi
- import numpy as np : alias court, convention universelle
- from math import sqrt : on appelle sqrt directement

ÉCOSYSTÈME PYTHON

Commandes Essentielles : pip install

pip install requests

1. Installation simple

Télécharge un paquet depuis PyPI.

pip install requests==2.31.0

2. Version spécifique

Fixe une version précise du paquet.

pip install --upgrade requests

3. Mise à jour

Maintient le paquet à sa dernière version.

pip install numpy pandas

4. Paquets multiples

Installe plusieurs paquets simultanément.

pip install -r requirements.txt

5. Fichier de dépendances

Déploie la liste complète des paquets d'un projet.
Dans les projets Python, on utilise souvent un fichier texte contenant la liste de tous les paquets nécessaires

Astuce Système (macOS / Linux) : Si plusieurs versions de Python sont installées, privilégiez **python -m pip install nom_du_paquet**.

Exercices de la partie 1

À réaliser dans votre notebook, cellule par cellule. Les corrections sont revues à la séance suivante.

1. **Créer le notebook**
   Une cellule Markdown avec un titre, votre nom et la date.
2. **Variables et f-string**
   Stocker un nom, un âge, une moyenne, puis afficher une phrase complète.
3. **Liste de 6 notes**
   Calculer moyenne, minimum, maximum, et compter les notes supérieures ou égales à 10 avec une boucle.
4. **Dictionnaire d'un étudiant**
   Créer le dictionnaire, ajouter une clé « filiere », puis afficher tous les couples clé-valeur.
5. **Fonction mention(note)**
   Renvoyer « Très bien », « Bien », « Passable » ou « Ajourné » selon la note (barème au choix).
6. **Compréhension de liste**
   Appliquer mention à la liste de notes avec [mention(n) for n in notes].

Astuce : exécutez chaque cellule après l'avoir écrite pour repérer l'erreur tout de suite.

Exercice 1

Question 1

Markdown Jupyter

En-tête documenté et mise en forme.

Question 2

Variables & f-strings

Incrustation dynamique de variables.

Question 3

Analyse de Liste

Moyenne, min/max et comptage par boucle.

Question 4

Dictionnaires

Clés, valeurs et parcours .items().

Question 5

Fonction Conditionnelle

Calcul de la mention selon la note.

Question 6

Compréhension de Liste

Transformation concise de données.

EXERCICE q1 & q2

Markdown & Formatage f-string

1 • CELLULE MARKDOWN

En-tête de Document

```
# Exercices Q1 **Nom :** Sara **Date :** 11 octobre 2026
```

2 • SYNTAXE F-STRING

Affichage Dynamique

Une **f-string** (ou *formatted string literal*) est une méthode en Python permettant de **facilement intégrer des variables et des expressions à l'intérieur d'une chaîne de caractères** (du texte).

Pour créer une f-string, il suffit de placer la lettre **f** juste devant les guillemets, puis d'insérer les variables entre **accolades {}** :

```
nom = "Sara"
age = 21
moyenne = 14.5
print(f"L'étudiant(e) {nom} a {age} ans et a obtenu une moyenne de {moyenne}/20.")
```

Sortie : L'étudiant(e) Sara a 21 ans et a obtenu une moyenne de 14.5/20.

EXERCICE q3

Traitement de Liste : Calculs & Boucle for

```
notes = [12, 8, 15, 9, 17, 10]

# Calculs
moyenne_notes = sum(notes) / len(notes)
note_min = min(notes)
note_max = max(notes)

# Compter les notes >= 10 avec une boucle
compteur = 0
for note in notes:
    if note >= 10:
        compteur += 1

print(f"Notes : {notes}")
print(f"Moyenne : {moyenne_notes:.2f}")
print(f"Note minimale : {note_min}")
print(f"Note maximale : {note_max}")
print(f"Nombre de notes >= 10 : {compteur}")
```

EXERCICE q4

Dictionnaire & Iteration .items()

```
# Création et modification

etudiant = {"nom": "Sara", "age": 21}
etudiant["filiere"] = "Maths"

# Parcours des éléments
for cle, valeur in etudiant.items():
                print(cle, "->", valeur)
```

Résultat dans la Console

```
nom -> Sara
age -> 21
filiere -> Maths
```

EXERCICE q5

Fonctions & Conditions if / elif / else

```
def mention(note):
    if note >= 16:
        return "Très bien"
    elif note >= 14:
        return "Bien"
    elif note >= 10:
        return "Passable"
    else:
        return "Ajourné"

# Test rapide de la fonction
print(mention(15))
print(mention(15))
```

mention(15)

→ **"Bien"**

mention(8)

→ **"Ajourné"**

EXERCICE q6

Compréhension de Liste

```
# Application de la fonction mention() sur toute la liste notes
mentions_liste = [mention(n) for n in notes]
print(f"Notes : {notes}")
print(f"Mentions : {mentions_liste}")
```

mentions_liste = [mention(n) for n in notes]

- **Compréhension de liste (*list comprehension*)** : C'est une façon concis et élégante de construire une nouvelle liste en une seule ligne.
- **Déroulement** :
  - → for n in notes : Parcourt chaque note n de la liste notes.
  - → mention(n) : Applique la fonction mention() à la note n pour obtenir son résultat ("Passable", "Bien", etc.).
  - → [...] : Rassemble tous ces résultats dans une nouvelle liste enregistrée dans la variable mentions_liste.

Résultat Final :

Notes : [12, 8, 15, 9, 17, 10]

Mentions : ['Passable', 'Ajourné', 'Bien', 'Ajourné', 'Très bien', 'Passable']